

데이터 지향 설계와 명시적 GPU 제어를 위한 멀티플랫폼 C++ 프레임워크 dy_framework

팀명	DyD	학과	컴퓨터소프트웨어공학과	지도교수	정석용
팀원	한재승(팀장), 윤훈, 정현진, 정준혁				

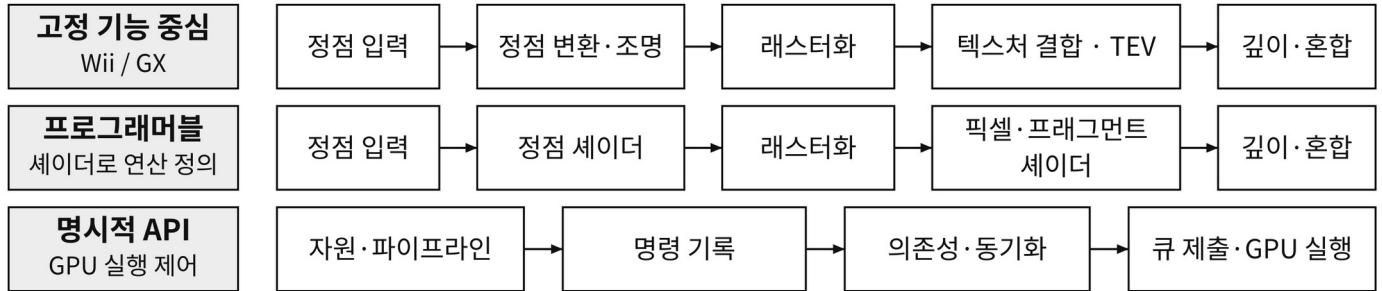
1. 작품의 개요 — 개발 배경과 필요성

GPU의 성능을 활용하는 출발점, 데이터와 실행 흐름의 설계

GPU는 그래픽스와 함께 인공지능의 학습·추론, 과학 시뮬레이션 등 대규모 연산을 수행하는 핵심 하드웨어로 활용되고 있다. 이러한 활용의 확대는 소프트웨어에도 하드웨어의 병렬 처리 능력과 메모리 구조를 고려하는 설계를 요구한다. CPU 캐시와 GPU 병렬 처리에 맞춰 데이터의 배치·접근·변환을 우선하는 데이터 지향 설계(DOD)는 하드웨어의 성능을 활용하기 위한 중요한 설계 관점이다.[1-3]

현대 그래픽스의 설계 변화와 멀티플랫폼 구현의 과제

그래픽스는 고정된 처리 기능을 설정하는 방식에서 셰이더로 연산을 정의하는 방식으로 발전해 왔다. 현대의 명시적 API에서는 GPU 자원의 배치와 사용 시점, 작업 간 동기화와 명령 제출까지 개발자가 설계하는 대상으로 확대되었다.[4,5,7]



위 두 행: GPU 처리 단계의 변화 / 아래 행: 작업을 구성·제출하는 API의 제어 범위

그림 1. Wii는 고정 기능 중심 GPU를 사용한 대표적인 후기 기기다. TEV에는 구성 가능한 연산 단계가 포함된다.[6]

기존의 공통 API와 암묵적 자원 관리에 맞춰 작성한 통합 코드를 그대로 옮기는 것만으로는 이러한 제어 모델을 충분히 활용하기 어렵다. Vulkan·Direct3D 12·Metal의 서로 다른 자원·바인딩·명령 체계에 대응하면서, 공통으로 작성하던 코드도 API·플랫폼별 구현으로 다시 갈라진다. 현대 GPU의 제어 능력을 활용하면서 플랫폼마다 반복되는 구현을 줄일 수 있는 개발 기반이 요구된다.

dy_framework는 이러한 구현 차이를 공통 RHI에서 다루고, 데이터 지향 설계와 명시적 GPU 제어를 연결하는 C++ 프레임워크이다. 데이터의 준비와 처리부터 GPU 자원과 실행 명령까지 일관된 구조로 구성하여, 여러 플랫폼에서 데이터와 실행 흐름을 직접 설계할 수 있도록 한다.

2. 작품의 구성

데이터 처리, 작업 계획, 공통 RHI와 플랫폼별 구현으로 역할을 나눴다. RenderGraph는 패스의 의존성과 기록 순서를 구성하고, RHI는 명령 제출과 GPU 완료까지의 자원 보유를 맡는다.

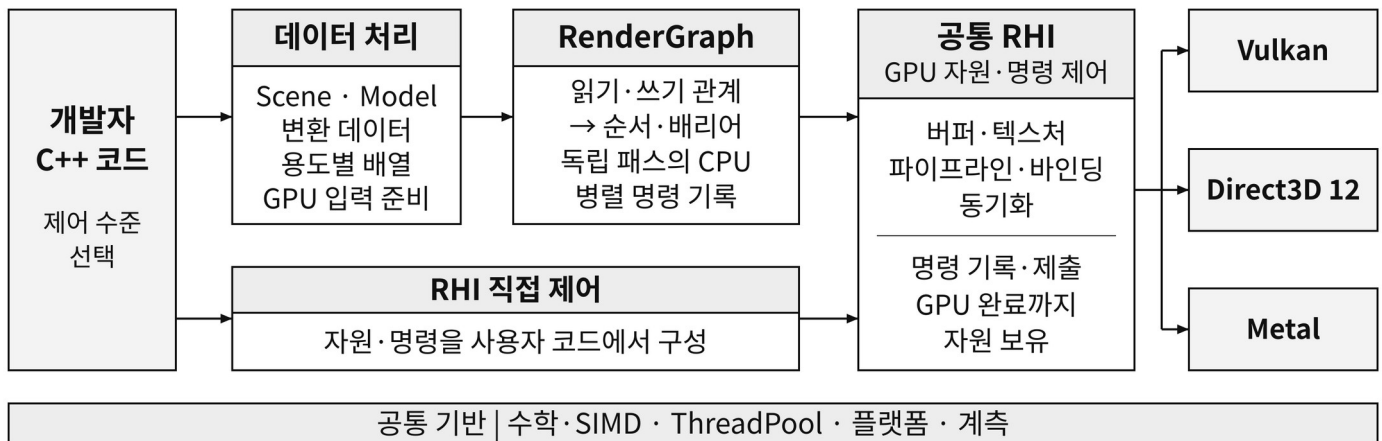


그림 2. 상위 데이터 경로와 RHI 직접 제어를 제공하며, 공통 기반이 각 계층을 지원한다.

3. 프로젝트 개발환경 및 개발자

개발도구 및 사용 언어 C++17 · CMake MSVC / GCC / Clang HLSL · GLSL · Metal Windows · Linux · macOS	사진 삽입 한재승 Core · RHI	사진 삽입 윤훈 Vulkan	사진 삽입 정현진 Direct3D 12	사진 삽입 정준혁 Metal
--	---	--------------------------------------	--	--------------------------------------

4. 핵심기술 설명

4-1. 데이터 지향 설계(DOD)에서 명시적 GPU 제어로

처리할 데이터의 배치·접근·변환을 먼저 설계하고, 그 흐름을 GPU 자원과 실행 명령으로 연결한다.

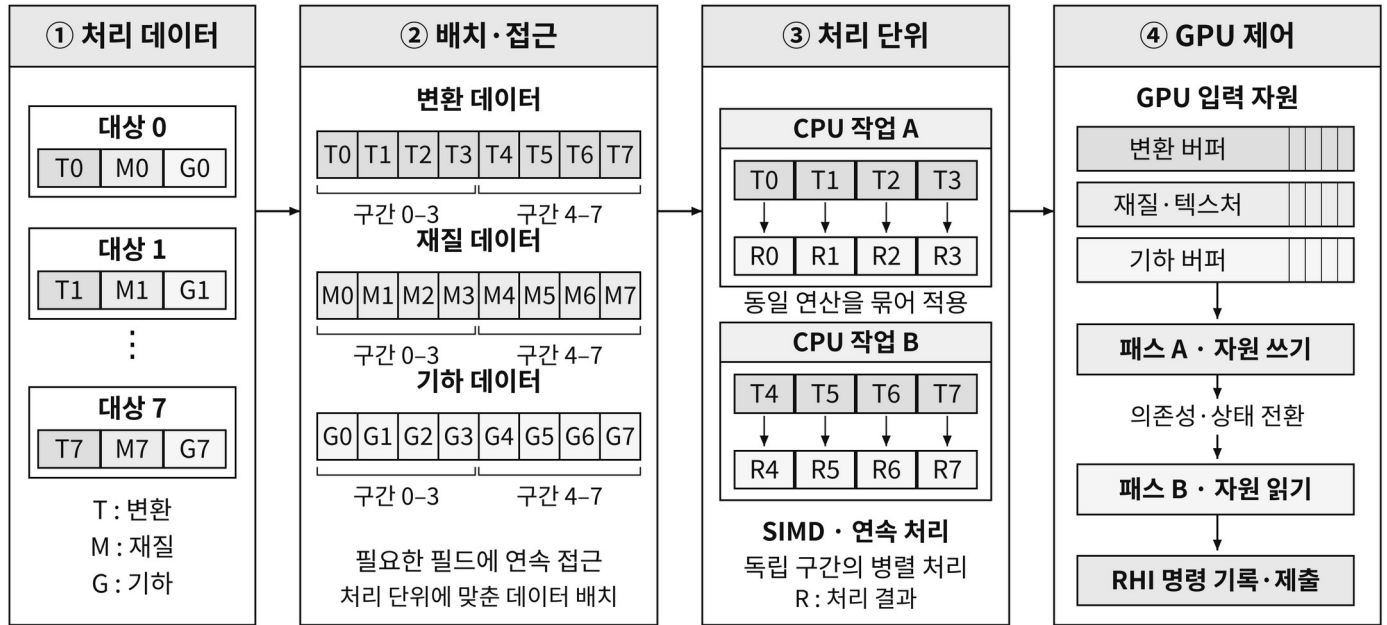


그림 3. 데이터 배열 → 독립 구간의 처리 → GPU 자원·의존성 구성. 배열과 처리 구간은 설계 관점을 보여주는 예시다.

4-2. 명시적 GPU 제어부터 고급 렌더링까지

자원의 배치와 접근, 작업의 의존성과 제출을 함께 다루는 관점은 GPU 기반 처리와 렌더링 기법으로 이어진다.

GPU 실행·데이터 제어	GPU 기반 처리·렌더링
RenderGraph / Resource Barriers 자원의 읽기·쓰기 관계를 바탕으로 작업 의존성과 상태 전환을 구성한다.	Indirect Drawing / GPU Culling GPU가 가시성을 판정하고 그리기 인자를 구성하여, 처리 대상을 GPU 실행 흐름 안에서 결정한다.
Multithread Command Recording 독립 작업의 RHI 명령과 네이티브 GPU 명령을 여러 CPU 스레드에서 병렬 기록하고, 의존 순서에 맞춰 제출한다.	Mesh Shader 작업 그룹 단위로 정점과 프리미티브를 생성·처리하여 기하 처리 흐름을 구성한다.
Instancing & Batching 공유 기하와 객체별 속성 배열을 활용하고, 호환되는 작업을 묶어 CPU의 반복 제출을 줄인다.	Meshlet Culling / LOD 메시를 작은 묶음으로 나누고 가시성과 상세도를 선택해 처리할 기하 데이터의 범위를 줄인다.
Bindless Materials 리소스 집합의 바인딩을 유지하면서 셰이더의 인덱스로 재질·텍스처를 선택한다.	Clustered Lighting 공간을 클러스터로 나누고 광원을 분류해, 각 영역에 필요한 광원 데이터를 참조한다.
Async Compute 그래픽스와 컴퓨트 작업을 별도 큐에 배치하고, 의존성을 조정해 실행 중첩을 구성한다.	Temporal Anti-Aliasing 이전 프레임 정보를 재투영·누적해 시간에 걸쳐 영상의 계단 현상을 줄인다.
	Volumetric Fog 시선 경로의 산란과 감쇠를 계산해 공간 속 빛과 안개를 표현한다.

주요 이론서: Real-Time Rendering, 4th Edition[6]

파이프라인·셰이딩·영상 처리·최적화의 관점을 GPU 자원과 실행 명령의 구성에 연결한다.

참고 자료 · 설계 관점과 렌더링 이론

- [1] Mike Acton. Data-Oriented Design and C++. CppCon, 2014.
- [2] Stoyan Nikolov. OOP Is Dead, Long Live Data-oriented Design. CppCon, 2018.
- [3] Timur Doumler. Want fast C++? Know your hardware! CppCon, 2016.
- [4] Microsoft. Important Changes from Direct3D 11 to Direct3D 12.
- [5] Khronos. Vulkan Specification — Synchronization and Cache Control.
- [6] Akenine-Möller et al. Real-Time Rendering, 4th ed. A K Peters/CRC Press, 2018.
- [7] Apple. Metal — MTLHazardTrackingMode and resource synchronization.